University of Toronto
Scarborough Campus

October 17, 2024

CSCC73 Midterm Review Seminar

Host: Christopher Nathanael

Duration: Two hours

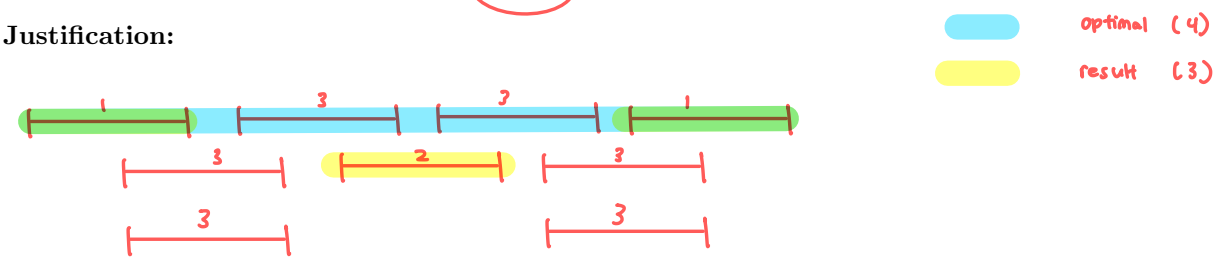
QUESTION 1.

For each of the statements below, indicate whether it is true or false by circling the appropriate response, and give a brief but convincing justification for your answer. Correct answer with ***justification left blank*** is worth more points than correct answer with wrong justification.

a. In the interval scheduling problem, sorting the intervals by increasing number of conflicts instead of increasing finish time would still give us the optimal solution

Circle the correct answer: **True** / **False**

Justification:



b. In the Minimum Lateness Scheduling problem, if all jobs have the same deadline, there is no need to sort the jobs in any particular order before greedily scheduling them, as any order will produce an optimal solution with the minimum maximum lateness.

Circle the correct answer: **True** / **False**

Justification:

The finish time of the last job in any order would be $\sum t(i)$

because the deadline is the same for all jobs

the max lateness would always be $\sum t(i)$ - deadline

c. In Huffman's algorithm, if there is exactly one symbol with frequency 1/3, all other symbols have frequency **strictly** less than 1/3, and the algorithm may produce a codeword of length 1.

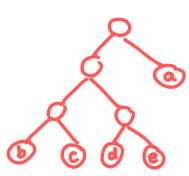
Circle the correct answer: **True** / **False**

Justification:

Example : define $\Gamma = \{a, b, c, d, e\}$ define frequency $f(i) = \begin{cases} 1/3, & i = a \\ 1/6, & i \neq a \end{cases}$

note : freq has to sum up to 1

Huffman Output:



[continued on the next page]

d. In Huffman's algorithm, if every symbol has frequency **strictly** less than 1/3, the algorithm may produce a codeword of length 1.

Circle the correct answer: **True** / **False**

Justification:

Sps $\exists \Gamma$ s.t $\forall i \in \Gamma$, $f(i) < \frac{1}{3}$ & algo produce codeword of length 1

Let a be this codeword of length 1

Case 1: Γ only has 2 symbols

$f(a) + f(b) < \frac{1}{3} + \frac{1}{3} = \frac{2}{3} \neq 1$, contra!

Case 2: Γ has >2 symbols

Consider 2nd last iter. of Huffmann's algo

leaf a, leaf/tree x, leaf/tree y

By sps. we know $f(a) \geq f(x)$ and $f(a) \geq f(y)$

However $f(a) < \frac{1}{3}$

$\therefore f(a) + f(x) + f(y)$

$\leq f(a) + f(a) + f(a)$

$< \frac{1}{3} + \frac{1}{3} + \frac{1}{3} = 1$

Contra!

Sum of freq in this iteration $\neq 1$

e. Suppose we are given an array A of n distinct elements, and we want to find $n/2$ elements in the array whose median is also the median of A. Any algorithm that does this must take $\Omega(n \log n)$ time.

Circle the correct answer: **True** / **False**

Justification:

We can use D-SELECT which only takes $\Theta(n)$ at worst

① Find median with D-SELECT $\Theta(n)$

② Partition array around median

③ take $n/4$ elements from either side to get a total of $n/2$ elements

f. If P is a shortest path from node u to node v in a directed graph with positive edge weights, then P is also a shortest path from u to v if we reduce the weight of each edge by 1.

Circle the correct answer: **True** / **False**

Justification:

QUESTION 2.

Your manager has been assigning you tedious tasks and making your workdays unbearable, so you decide it's time for a little bit of revenge (don't worry, it's just hypothetical... or is it?). The investment company you work for has stock prices for the next n consecutive days. Each day has a fixed stock price, and you have access to all this information.
Leveraging your advanced CSCC73 knowledge, you realize you can subtly sabotage the company's finances. The plan? Buy stocks on one day and sell them on a later day to make the **maximum loss**.
But there's a catch! You need to figure out the right days to buy and sell in $\mathcal{O}(n \log n)$ time before your colleagues get wind of your diabolical scheme.

Input:
An array $A[1..n]$ of integers, where $A[i]$ represents the stock price on day i.

Output:
The pair of days (i, j), where $i < j$, such that buying on day i and selling on day j results in the **maximum possible loss**. If no such loss can be made, return (i, j) with a loss of zero.

Explain the main idea(s) of your algorithm, describe the algorithm in pseudocode, and analyze its running time. Do not justify its correctness. Your algorithm must run in $\mathcal{O}(n \log n)$ time.

MAIN IDEA AND PSEUDOCODE:

3 cases:
i. Best loss in first half
ii. Best loss in second half
iii. Best loss crosses both halves

Information needed to calculate i, ii, iii
buy_date, sell_date, max_date, min_date

Base case:
size of 1

```
maxLoss (A[1...n])
    if len(A) = 1
        return [1, 1, 1, 1]
    mid = ⌊n/2⌋
    l_buy, l_sell, l_max, l_min = maxLoss (A[1:mid])
    r_buy, r_sell, r_max, r_min = maxLoss (A[mid+1:n]) + (mid, mid, mid, mid)
    l_loss = A[l_buy] - A[l_sell]
    r_loss = A[r_buy] - A[r_sell]
    cross_loss = A[l_max] - A[r_min]
    if l_loss ≤ r_loss and l_loss ≤ cross_loss
        buy = l_buy
        sell = l_sell
    if r_loss ≤ l_loss and r_loss ≤ cross_loss
        buy = r_buy
        sell = r_sell
    if cross_loss ≤ r_loss and cross_loss ≤ l_loss
        buy = l_min
        sell = r_max
    max = max(l_max, r_max)
    min = min(l_min, r_min)
    return (buy, sell, max, min)
```

```
Wrapper(A):
    a b c d = maxLoss(A)
    return a, b
```

RUNNING TIME ANALYSIS:

$T(n) = 2T(n/2) + c$

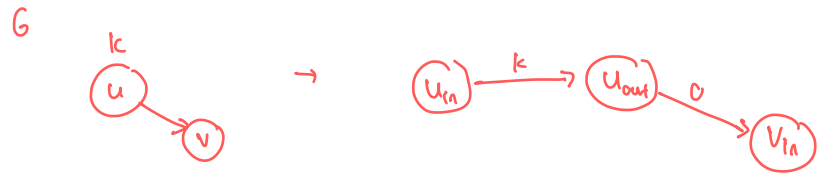
$a = 2$
$b = 2$ $\Rightarrow$ $a > b^d$ $\Rightarrow$ $\Theta(n)$
$d = 0$

QUESTION 3.

Give an algorithm that takes as input an **unweighted** directed graph $G = (V, E)$, where each **node** $v \in V$ has a weight $w(v)$, each **edge** $e \in E$ has 0 weight and returns the minimum weight path from a source **node** s to a target **node** t. The weight of a path is the sum of the weights of the **nodes** on the path (including the start and end **nodes**).

Explain the main idea(s) of your algorithm, describe the algorithm in pseudocode, and analyze its running time. Do not justify its correctness. For full credit, your algorithm should run in $\mathcal{O}((m + n) \log n)$ time where n be the number of nodes and m be the number of edges. Assume that the graph is given in adjacency list form.

MAIN IDEA AND PSEUDOCODE:



Reduction :

For each node u in G ,

create 2 nodes u_{in} and u_{out} in G' + edge (u_{in}, u_{out}) with weight = weight of u

For each edge (u,v) in G

create edge (u_{out}, v_{in}) with weight 0 in G'

Run Dijkstra from s_{in} to t_{out} in G'

RUNNING TIME ANALYSIS:

Constructing G' takes $\Theta(n+m)$ time

G' has 2n nodes , n+m edges

Dijkstra takes $\Theta((2n+n+m) \log(2n)) = \Theta((m+n) \log n)$

QUESTION 4.

You are given an unlimited supply of coins of limited denominations $x_1, x_2, \ldots, x_n$, and you wish to make change for a target value v. That is, you want to find a set of coins whose total value is exactly v. Note that it may not always be possible; for instance, if the denominations are 3, 5, and 7, you can make change for 16 but not for 22.

In this question, you are to develop a pseudo :) polynomial-time **dynamic programming** algorithm that determines whether it is possible to make change for a value v using the available coin denominations $x_1, x_2, \ldots, x_n$.

a. Define clearly the subproblems that your dynamic programming algorithm will solve.

Answer:

$D(i)$ = is it possible to make a sum of i

b. Give a recursive formula to compute the solutions to the subproblems in part (a). No justification of correctness is needed.

Answer:

$$D(i) = \begin{cases} \text{True} & , i \in [x_1, \ldots, x_n] \\ \text{False} & , i < \min\{x_1, \ldots, x_n\} \\ D(i - x_1) \text{ or } \ldots \text{ or } D(i - x_n) & , \text{else} \end{cases}$$

[continued on the next page]

c. Describe your dynamic programming algorithm in pseudocode. Do not explain why your algorithm is correct.

Answer:

```
D[1,2,4,6] = [F,F,F,F]
D[0,3,5,7] = [T,T,T]
if v < 8 : return D[8]

for i from 8 to v :
    D[i] = D(i-3) or D(i-5) or D(i-7)

return D[n]
```

```
make change (V, {x1, ..., xn})
    dp[] = [False, ..., False]   (V+1 times)

    dp[0, x1, ..., xn] = True

    for i = max{x1, ..., xn}+1 to V
        for xj in {x1, ..., xn}
            dp[i] = dp[i] or dp[i-xj]

    return dp[V]
```

d. Analyze the running time of your algorithm.

Answer:

take $\Theta(1)$

take $\Theta(Vn)$

$\therefore \Theta(Vn)$

QUESTION 5 (Taken from DPV 6.1).

A *contiguous subsequence* of a list S is a subsequence made up of consecutive elements of S. For instance, if S is

$$5, 15, -30, 10, -5, 40, 10,$$

then $15, -30, 10$ is a contiguous subsequence but $5, 15, 40$ is not. Give a linear-time algorithm for the following task:

Input: A list of numbers, $a_1, a_2, \ldots, a_n$.

Output: The contiguous subsequence of maximum sum (a subsequence of length zero has sum zero).

For the preceding example, the answer would be $10, -5, 40, 10$, with a sum of 55.

In this question, you are to develop a polynomial-time **dynamic programming** algorithm that determines the contiguous subsequence of maximum sum

a. Define clearly the subproblems that your dynamic programming algorithm will solve.

Answer:

We know that any cont. subsequence has to end at some index

Which one produces max sum? Need to get max of each index' max

(This sounds familiar to W5 tutorial q2.

$S(i)$ = max sum of cont. subseq ending at i

(solution to original problem : $\max \{ S(i) : 1 \leq i \leq n \} \cup \{0\}$)

b. Give a recursive formula to compute the solutions to the subproblems in part (a). No justification of correctness is needed.

Answer:

$$S(i) = \begin{cases} a_i & \text{if } i = 1 \\ \max\{ S(i-1) + a_i, a_i \} & \text{otherwise} \end{cases}$$

[continued on the next page]

c. Describe your dynamic programming algorithm in pseudocode. Do not explain why your algorithm is correct.

Answer:

```
max = a_1
end = 1
S[1] = a_1
for i from 2 to n
        S[i] = max { S(i-1) + a_i , a_i }
        if S[i] > max :
            max = S[i]
            end = i
Start = end
sum = 0
while start > 1
      sum += a[start]
      if sum == max :
          break
      start --
if max ≤ 0 :
    return [ ]
else
    return a[start : end]
```

d. Analyze the running time of your algorithm.

Answer:

Lines 4-8 take $\Theta(n)$

Lines 12-15 take $\Theta(n)$

The rest takes $\Theta(1)$

$\Rightarrow \Theta(n)$